INSTITUTO NACIONAL DE APRENDIZAJE

Núcleo Comercio y Servicios · Subsector Informática y Comunicación

Diseño de algoritmos

Sesión 20 · Arreglos, matrices y cadenas

UA2 · Programación estructurada | Módulo CSTI12010

Persona facilitadora

Giovanni Antonio Coto Calderón

SABER 1

¿Qué es un arreglo?

Un **arreglo** es un conjunto de casillas **del mismo tipo** con **un solo nombre**. Cada casilla se identifica con un **índice** que empieza en **0**: **double notas[5];** crea notas[0], notas[1], …, notas[4].

Mismo tipo

Todas int o todas double.

Índice desde 0

La última es N − 1.

Termina en N -1 donde N es el numero total de elementos y -1 por el cero del inicio. Evita error de desbordamiento.

Tamaño fijo

Se define al declarar.

Recorrido con for

for (i = 0; i < N; i++)

Declarar, inicializar y recorrer

Las llaves { } dan valores iniciales. Un **for** de **0** hasta **< N** visita cada casilla una vez.

arreglo.c

```c
#include <stdio.h>
int main(void) {
    // 5 casillas: de notas[0] a notas[4]
    int notas[5] = {80, 75, 90, 65, 100};
    int i;
    for (i = 0; i < 5; i++) {
        printf("notas[%d] = %d\n", i, notas[i]);
    }
    return 0;
}
```

```
<= 5: comportamiento indefinido. Resultado Numero
aleatorio en nota []5
```

Salida real (gcc)

```
notas[0] = 80
notas[1] = 75
notas[2] = 90
notas[3] = 65
notas[4] = 100
```

Promedio, mayor y menor

Con los datos guardados se pueden hacer **varios recorridos**. Para el mayor y el menor se empieza con **la primera casilla**.

estadisticas.c

```c
#include <stdio.h>
#define N 5
int main(void) {
    double notas[N] = {80, 75, 90, 65, 100};
    double suma = 0, mayor, menor;
    int i;
    mayor = menor = notas[0];       // se empieza con la primera
    for (i = 0; i < N; i++) {
        suma += notas[i];
        if (notas[i] > mayor) { mayor = notas[i]; }
        if (notas[i] < menor) { menor = notas[i]; }
    }
    printf("Promedio: %.2f\n", suma / N);
    printf("Mayor: %.1f  Menor: %.1f\n", mayor, menor);
    return 0;
}
```

Salida real (gcc)

```
Promedio: 82.00
Mayor: 100.0  Menor: 65.0
```

CUIDADO

Fuera de los límites

C **no revisa** los índices: notas[5] en un arreglo de 5 **compila sin advertencias** con -Wall y lee memoria ajena.

Incorrecto

i llega a 5

```
int notas[5] = {80, 75, 90, 65, 100};
for (i = 0; i <= 5; i++) {
    printf("%d\n", notas[i]);
}
// La ultima vuelta lee notas[5]:
// valor impredecible o cierre
```

Correcto

i va de 0 a N - 1

```
#define N 5
int notas[N] = {80, 75, 90, 65, 100};
for (i = 0; i < N; i++) {
    printf("%d\n", notas[i]);
}
// Siempre < N
```

Buscar un dato en un arreglo

La **búsqueda lineal** revisa casilla por casilla. **-1** significa «no encontrado»; **break** detiene la búsqueda al encontrarlo.

buscar.c (fragmento)

```c
int codigos[N] = {101, 205, 310, 412, 518};
int buscado = 310, posicion = -1, i;
for (i = 0; i < N; i++) {
    if (codigos[i] == buscado) {
        posicion = i;
        break;              // ya se encontro
    }
}
if (posicion >= 0) {
    printf("Codigo %d en la posicion %d\n", buscado, posicion);
} else {
    printf("Codigo %d no existe\n", buscado);
}
```

Salida real (gcc)

```
Codigo 310 en la posicion 2
```

SABER 2

Matrices: filas y columnas

Una **matriz** tiene dos índices: **ventas[fila][columna]**. Se recorre con los **for anidados** de la sesión 19.

matriz.c

```c
#include <stdio.h>
int main(void) {
    // 2 sucursales (filas) x 3 dias (columnas)
    int ventas[2][3] = { {120, 95, 130},
                         {80, 110, 90} };
    int f, c, total;
    for (f = 0; f < 2; f++) {
        total = 0;
        for (c = 0; c < 3; c++) {
            printf("%5d", ventas[f][c]);
            total += ventas[f][c];
        }
        printf("  | total %d\n", total);
    }
    return 0;
}
```

Salida real (gcc)

```
  120   95  130  | total 345
   80  110   90  | total 280
```

Caracteres y cadenas

Arreglo de char

Letras sueltas

- char letras[3] = {'A', 'n', 'a'};
- 3 casillas, sin marca de fin.
- No se puede mostrar con %s.
- Se recorre letra por letra.

Cadena

Texto terminado en '\0'

- char nombre[30] = "Ana";
- Ocupa 4 casillas: A, n, a, '\0'.
- El '\0' marca dónde termina.
- Se muestra con %s y se procesa con string.h.

Leer textos con espacios: fgets

fgets(cadena, tamaño, stdin) lee la **línea completa** sin pasarse del tamaño. Guarda también el **Enter**, que se reemplaza por **'\0'**.

fgets.c

```c
#include <stdio.h>
#include <string.h>
int main(void) {
    char nombre[40];
    printf("Nombre completo: ");
    fgets(nombre, sizeof(nombre), stdin);
    // fgets guarda tambien el Enter: se cambia por '\0'
    nombre[strcspn(nombre, "\n")] = '\0';
    printf("Hola, %s!\n", nombre);
    printf("Caracteres: %d\n", (int) strlen(nombre));
    return 0;
}
```

Salida real (Ana Mora Solis)

```
Nombre completo: Ana Mora Solis
Hola, Ana Mora Solis!
Caracteres: 14
```

Funciones de cadenas

Función	Qué hace	Ejemplo	Resultado
`strlen(s)`	Cuenta caracteres (sin '\0')	`strlen("Ana Mora")`	`8`
`strcpy(d, s)`	Copia s en d	`strcpy(copia, "Ana")`	`"Ana"`
`strcat(d, s)`	Agrega s al final de d	`strcat(copia, " Mora")`	`"Ana Mora"`
`strcmp(a, b)`	Compara: 0 si son iguales	`strcmp("admin", "admin")`	`0`
`strcmp(a, b)`	< 0 si a va antes	`strcmp("Ana", "Luis")`	`negativo`
`strcspn(s, c)`	Posición del primer c	`strcspn("Hola\n", "\n")`	`4`

Las cadenas **no** se asignan con **=** ni se comparan con **==**: se usan **strcpy** y **strcmp**. El destino debe tener **espacio suficiente**.

strcpy, strcat, strlen y strcmp

cadenas.c

```c
#include <stdio.h>
#include <string.h>
int main(void) {
    char nombre[30] = "Ana";
    char copia[30];
    strcpy(copia, nombre);          // copia = "Ana"
    strcat(copia, " Mora");         // copia = "Ana Mora"
    printf("copia: %s\n", copia);
    printf("strlen: %d\n", (int) strlen(copia));
    printf("iguales: %d\n", strcmp(nombre, "Ana") == 0);
    printf("Ana antes que Luis: %d\n", strcmp("Ana", "Luis") < 0);
    return 0;
}
```

Salida real (gcc)

```
copia: Ana Mora
strlen: 8
iguales: 1
Ana antes que Luis: 1
```

getchar y putchar

getchar() lee **un carácter**; **putchar(c)** escribe uno. Sirven para procesar un texto letra por letra.

caracteres.c (fragmento)

```c
int c;                          // int: getchar puede dar EOF
int noEspacios = 0, espacios = 0;
printf("Escriba una frase: ");
c = getchar();                  // lee UN caracter
while (c != '\n' && c != EOF) {
    if (c == ' ') {
        espacios++;
    } else {
        noEspacios++;
    }
    putchar(c);                 // lo escribe de nuevo
    c = getchar();
}
printf("\nOtros: %d  Espacios: %d\n", noEspacios, espacios);
```

Salida real (Hola INA 2026)

```
Escriba una frase: Hola INA 2026
Hola INA 2026
Otros: 11  Espacios: 2
```